

PYTHON DAY 1

1. Python in Data Engineering

Data engineering means moving data from one place to another and cleaning it on the way. Python is the tool that does the work in the middle.

The flow in your document works like this:

1. **Receive File:** a CSV arrives, for example customers.csv.
2. **Read Data:** Python opens it.
3. **Validate:** it checks the data is correct (no missing IDs, amounts are numbers).
4. **Remove Bad Records:** it throws out rows that fail the checks.
5. **Transform:** it reshapes the data, such as changing formats or adding columns.
6. **Write Output:** it saves the clean file.
7. **S3 → Glue/PySpark → Redshift:** the clean file is stored in S3, processed at scale by Glue, and loaded into the Redshift warehouse.

2. print()

print() shows something on the screen. Whatever is inside the brackets gets displayed.

```
print("Hello Data Engineering")
```

- Text goes inside **quotes** " ". Without quotes Python thinks it's a variable name and gives an error.
- Each print() writes on a **new line**.
- Pipeline programs use print() as a simple log, such as "Pipeline Started" and "Processing completed".

3. Variables

A variable is a **labelled box** that holds a value, so you can use it again.

```
source_file = "customers.csv"
```

Part	In the example	Meaning
Name	source_file	the label on the box
=	=	"put this value into the box" (not "equals" as in maths)
Value	"customers.csv"	what's inside

Naming rules:

- Allowed: letters, numbers and _. Use customer_name, not customer name.
- It cannot **start** with a number, so 1name is wrong.
- No spaces.
- It is **case-sensitive**, so name and Name are two different variables.
- Convention: lowercase with underscores (total_amount). This is called snake_case.

4. Basic Data Types

Every value has a type, and the type decides what you can do with it.

Type	Meaning	Example
str	text	"Arun"
int	whole number	101
float	decimal number	25000.50
bool	True or False	True

The dataset table in your document shows how a real file maps to Python types:

- customer_id → int
- customer_name → str
- amount → float
- active → bool

5. Type Conversion

This is one of the most important ideas today. Data read from files (CSV, JSON, APIs) often arrives as **text**, even when it looks like a number.

```
quantity = "10"
price = "500"
print(quantity + price) # 10500
```

With strings, + **joins** them, so "10" + "500" becomes "10500". It doesn't calculate anything. To get the real answer, convert first:

```
total = int(quantity) * int(price) # 5000
```

Function	Does
int("100")	text → whole number
float("500.50")	text → decimal
str(100)	number → text
bool(1)	→ True (0 gives False)

Two things to watch:

- int("abc") causes an **error**, because the text must actually look like a number.
- int("5000.50") also errors. Use float() first for decimals.

Practice: a = "20" and b = "30". Print their sum as the number 50, not "2030".

Answer: print(int(a) + int(b))

6. Operators

Operator	Meaning	10 and 3
+	add	13
-	subtract	7
*	multiply	30
/	divide	3.333...
%	remainder	1
**	power	1000

Note that 10 / 5 gives 2.0, not 2. In Python, / always returns a float.

The pipeline calculation in your document is a very common data engineering task:

```
successful_records = total_records - failed_records # 950
success_percentage = (successful_records / total_records) * 100 # 95.0
```

Brackets matter. Python does the part inside () first.

Practice: If 400 of 2000 records failed, what is the failure percentage?

Answer: $(400 / 2000) * 100$ gives 20.0.

7. Strings

Strings are used constantly for file names, S3 paths and column names.

- **Joining** uses +: first_name + " " + last_name gives Arun Kumar. The " " adds the space between.
- **Methods** are actions you attach with a dot:

Method	Result on example
"aws".upper()	AWS
"AWS".lower()	aws
"aws data".title()	Aws Data
" Arun ".strip()	Arun (spaces removed from both ends)
"a.csv".replace(".csv", ".json")	a.json
file.startswith("customer")	True/False
file.endswith(".csv")	True/False

Why strip() matters: the values "Arun" and " Arun " look the same to you but are different to Python. Stray spaces cause many data bugs, so cleaning with strip() is a daily habit.

Why endswith() matters: it's how a pipeline checks "is this really a CSV file?" before processing it. Methods don't change the original variable. They *return* a new value, so save the result if you want to keep it: new_name = file_name.replace(...).

Practice: file_name = " Sales_Report.CSV ". Clean the spaces, convert to lowercase, and check whether it ends with .csv.

```
file_name = file_name.strip().lower()
print(file_name.endswith(".csv")) # True
```

8. input()

input() pauses the program and waits for the user to type something.

```
name = input("Enter your name: ")
```

The text in brackets is the prompt shown to the user. **input()** always returns a string, even if the user types 25. That's why section 5 matters again:

```
record_count = int(input("Enter record count: "))
```

Here input() runs first, then int() converts the result.

Practice: Ask for two numbers and print their total.

```
a = int(input("First number: "))
b = int(input("Second number: "))
print("Total:", a + b)
```